PyMISP with MISP 2.5

Build models, query context, automate deliberate changes

CIRCL / MISP Project

https://www.misp-project.org/

Reference snapshot: 2 November 2026



From a notebook to a deliberate integration

- Build and inspect models without contacting a server.
- Map Python operations to the MISP 2.5 REST API.
- Query the right scope and recognise incomplete results.
- Create, verify and remove a synthetic training event.

From a notebook to a deliberate integration
- Build and inspect models without contacting a server.
- Map Python operations to the MISP 2.5 REST API.
- Query the right scope and recognise incomplete results.
- Create, verify and remove a synthetic training event.

Allow 40–50 minutes for the slides and 30–40 minutes for the workshop. The companion notebook runs offline by default. The README links the REST API and object-template courses, pins source revisions and describes exactly which checks were executed. Notebook examples are adapted from the upstream FullOverview and SearchFullOverview tutorials and the archived Introduction to PyMISP workshop, with current API semantics and synthetic data.

Client, models and API

Pin the client and identify the server

- Reviewed: MISP 2.5.50 and PyMISP 2.5.34.4.
- PyMISP requires Python 3.10 or later, below 4.0.
- PyPI and GitHub release checked on 2 November 2026.
- Server and client version numbers need not match.

Pin the client and identify the server
- Reviewed: MISP 2.5.50 and PyMISP 2.5.34.4.
- PyMISP requires Python 3.10 or later, below 4.0.
- PyPI and GitHub release checked on 2 November 2026.
- Server and client version numbers need not match.

The installed release is 2.5.34.4, checked against PyPI metadata and the GitHub release. Its source revision is 45c906070c287dffeed353069cb3daddceab2ea3; MISP 2.5.50 is fc99c9e3fc85beddc6ef18835359f3c8b7f44608. This course uses Python 3.12. Consult the server's getVersion response and pymisp_recommended_version; current package does not imply every server supports every operation. Recheck releases before refreshing the pin.

Two complementary interfaces

- PyMISP: authenticated requests to the server.
- MISPEvent, MISPAttribute, MISPObject: local models.
- Model changes and JSON conversion do not save remotely.
- Use the public client methods to persist intended changes.

Two complementary interfaces
- PyMISP: authenticated requests to the server.
- MISPEvent, MISPAttribute, MISPObject: local models.
- Model changes and JSON conversion do not save remotely.
- Use the public client methods to persist intended changes.

The PyMISP constructor itself performs server requests for version, current-user/role/settings and attribute types. Do not put it in an offline cell. Creating a MISPEvent and adding attributes/objects are local operations. ExpandedPyMISP remains a compatibility class in this release; use PyMISP for new examples. Model helpers simplify encoding but do not bypass server validation or access control.

The Python methods still use REST routes

Python operation	REST route in this snapshot
`search_index(...)`	`POST events/index`
`search(...)` (attributes)	`POST attributes/restSearch`
`get_event(uuid)`	`GET events/view/{uuid}`
`add_event(event)`	`POST events/add`
`update_attribute(...)`	`POST attributes/edit/{uuid}`

The Python methods still use REST routes

Python operation | REST route in this snapshot
search_index(...) | POST events/index
search(...) (attributes) | POST attributes/restSearch
get_event(uuid) | GET events/view/{uuid}
add_event(event) | POST events/add
update_attribute(...) | POST attributes/edit/{uuid}

The table shows the ordinary calls used here. get_event can use POST when passing certain options. Additional methods map to objects/add, tags/attachTagToObject, sightings/add and events/delete. An HTTP POST is not necessarily a write: restSearch is a read operation. See the REST API course for raw request/response envelopes and the deployed instance's api/openapi and api/rest interfaces.

Authentication and visibility remain server decisions

- The raw auth key goes in the Authorization header.
- An API-enabled role and visible data are still required.
- Advanced keys can restrict IPs, expiry and read-only use.
- Read-only keys can POST searches; they cannot create events.

Authentication and visibility remain server decisions
- The raw auth key goes in the Authorization header.
- An API-enabled role and visible data are still required.
- Advanced keys can restrict IPs, expiry and read-only use.
- Read-only keys can POST searches; they cannot create events.

Obtain a permitted auth key through your own instance's profile/auth-key interface, rather than a hard-coded community host. Keys inherit account visibility and relevant role permissions, with additional advanced-key restrictions. A writer can get proposals instead of edits on records it does not own. Never use a site-admin result as evidence of ordinary-user visibility. The lab uses an ordinary writer and a read-only key; administrative provisioning is separate from learner exercises.

Install the reviewed release in a virtual environment

```
python3 -m venv .venv
. .venv/bin/activate
python -m pip install -r examples/requirements.txt
python -m unittest examples.test_examples -v
python -m examples.models
```


Install the reviewed release in a virtual environment
python3 -m venv .venv
. .venv/bin/activate
python -m pip install -r examples/requirements.txt
python -m unittest examples.test_examples -v
python -m examples.models

Run from a.2-pymisp. requirements.txt pins pymisp==2.5.34.4. The package's dependencies are resolved by pip; this is a course-version pin, not a complete transitive lockfile. Use python -m pip install –upgrade pymisp to assess a future release in a separate environment, then rerun checks and update the pin deliberately. File-analysis extras are optional and are not required for the synthetic domain-ip exercise. Avoid sudo pip and unpinned Git main installs as the classroom baseline.

Connect with trusted TLS and bounded request timeouts

```python
from pymisp import PyMISP
from examples.common import Config

config = Config.from_env()
misp = PyMISP(config.url, config.key,
              ssl=config.verify, timeout=(5, 60),
              debug=False)
```


Connect with trusted TLS and bounded request timeouts



Set MISP_URL and MISP_API_KEY outside committed code and notebooks. Config preserves a trailing slash, including a deployment subdirectory: this release's search uses urljoin. For a private CA, REQUESTS_CA_BUNDLE supplies a trusted CA bundle to ssl; otherwise verification remains True. HTTP is allowed only on loopback for an isolated disposable lab. Timeout is a connect/read tuple, not an overall workflow deadline. Requests retains normal environment proxy handling. Do not print the client or key.

Check the response before using it

```python
from pymisp import MISPEvent
from examples.common import expect_model

saved = expect_model(
    misp.get_event(event_uuid, pythonify=True),
    MISPEvent,
)
```


Check the response before using it
from pymisp import MISPEvent
from examples.common import expect_model
saved = expect_model(
misp.get_event(event_uuid, pythonify=True),
MISPEvent,
)

PyMISP can raise transport/server/decoding exceptions or return an errors dictionary. pythonify=True does not guarantee a typed success. Bulk attribute addition has its own attributes/errors result shape, and some denied single-attribute writes can fall back to proposals. The helpers check expected model types and action results, then the workshop refetches saved state. An empty list from a valid search is different from an error. Avoid printing full responses or verbose logs into shared notebooks.

Read the right data

Choose the search scope first

- `search_index`: event summaries from the index.
- `search(controller="events")`: matching events.
- `search(controller="attributes")`: matching attribute rows.
- `get_event(uuid)`: one known event and its context.

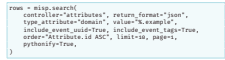
Choose the search scope first
- search_index: event summaries from the index.
- search(controller="events"): matching events.
- search(controller="attributes"): matching attribute rows.
- get_event(uuid): one known event and its context.

search defaults to the events controller. This released generic search method accepts events, attributes and objects; other endpoint families have dedicated methods. metadata=True on event search omits attributes and proposals; it is not the same as a full event with no attributes. Choose the controller to match the unit being consumed. Event and attribute limits count different things.

One typed attribute page

```
rows = misp.search(
    controller="attributes", return_format="json",
    type_attribute="domain", value="%.example",
    include_event_uuid=True, include_event_tags=True,
    order="Attribute.id ASC", limit=10, page=1,
    pythonify=True,
)
```


One typed attribute page



Use one_page from examples/read.py for the checked version. These are reserved synthetic .example domains. published and to_ids are deliberately omitted so ordinary JSON includes both states in the reviewed source. Attribute results can include attributes contained in objects. The code retrieves one page, not a complete export, and prints UUIDs/types rather than arbitrary incident values.

Raw JSON, normalised JSON and typed models differ

Interface	Attribute-search result
Raw HTTP JSON	`response.Attribute` array
PyMISP JSON	Dictionary containing `Attribute`
PyMISP with `pythonify=True`	List of `MISPAttribute` objects

Do not reuse the raw-envelope parser on typed results.

Raw JSON, normalised JSON and typed models differ
Interface | Attribute-search result
Raw HTTP JSON | response.Attribute array
PyMISP JSON | Dictionary containing Attribute
PyMISP with pythonify=True | List of MISPAttribute objects
Do not reuse the raw-envelope parser on typed results.

PyMISP's response checker unwraps the outer response field. Event searches have a different normalised shape. The notebook sends the equivalent bounded raw query and compares UUID sets with typed output on a stable fixture. pythonify=False does not restore the raw HTTP envelope. Large pythonified results use memory; retrieve bounded pages and retain only needed fields.

Translate parameter names and states deliberately

REST JSON	PyMISP keyword
`type`	`type_attribute`
`returnFormat`	`return_format`
`includeEventUuid`	`include_event_uuid`
`includeEventTags`	`include_event_tags`
`enforceWarninglist`	`enforce_warninglist`

Translate parameter names and states deliberately
REST JSON PyMISP keyword
type type_attribute
returnFormat return_format
includeEventUuid include_event_uuid
includeEventTags include_event_tags
enforceWarninglist enforce_warninglist

Use numeric 0/1 when restricting published or to_ids in these search examples; avoid assuming Python False or literal JSON false consistently selects state 0 on every endpoint. This PyMISP release rejects to_ids=[0,1], although the raw endpoint supports that array. Omit to_ids for both states in this ordinary JSON query. Unknown kwargs are forwarded, not necessarily validated: a misspelling may silently fail to constrain a query. The released-client contract tests exercise these translations.

A matching event can contain nonmatching attributes

- Event search selects events using the supplied filters.
- A full event result can include broader incident context.
- Attribute search selects the rows required by a consumer.
- Retain event UUID and inherited-tag context when needed.

A matching event can contain nonmatching attributes
- Event search selects events using the supplied filters.
- A full event result can include broader incident context.
- Attribute search selects the rows required by a consumer.
- Retain event UUID and inherited-tag context when needed.

Searching events for type domain does not mean every attribute in each full returned event is a domain. The synthetic fixture contains a domain, an object domain and an IP. Compare event and attribute search results without flattening them into an assumed equivalent set. Inherited event tags are context; they do not establish a direct attachment to the attribute. A tag filter is not an access-control grant.

Traverse bounded pages without claiming false completion

- Start at page 1 and use a stable order.
- A role cap can make every nonempty page short.
- For this ordinary JSON query, continue to an empty page.
- Fail on repeated UUIDs or the maximum-page bound.

Traverse bounded pages without claiming false completion
- Start at page 1 and use a stable order.
- A role cap can make every nonempty page short.
- For this ordinary JSON query, continue to an empty page.
- Fail on repeated UUIDs or the maximum-page bound.

collect_pages is deliberately restricted to the fixed ordinary JSON/SQL-filtered query in examples/read.py. It returns only after an empty page, and raises rather than returning a silently truncated result on repetition or the bound. It accumulates in memory only for this small workshop. It is not a generic exporter, streaming sync engine or database snapshot. Post-query warninglist/decay filters can create empty output pages before the end; do not add them to this loop unchanged.

Keep polling clocks and checkpoints explicit

```python
from time import time
upper = int(time())
lower = previous_checkpoint - overlap_seconds
# Use the same bounds for every page:
rows = misp.search(controller="attributes",
                   timestamp=[lower, upper],
                   limit=100, page=1, pythonify=True)
```


Keep polling clocks and checkpoints explicit
from time import time
upper = int(time())
lower = previous_checkpoint - overlap_seconds
Use the same bounds for every page:
rows = misp.search(controller="attributes",
timestamp=[lower, upper],
limit=100, page=1, pythonify=True)

This is an illustrative first page, not an executable sync client. Define previous_checkpoint and the initial backfill policy first. Attribute timestamp is the change clock at attribute scope; event_timestamp is the parent-event clock; publish_timestamp/last concerns publication; first_seen/last_seen are observation times. The timestamp window is inclusive. Upsert by UUID, overlap deliberately and advance the checkpoint only after every page and downstream write succeeds. Deletions, access loss and event-only context changes need reconciliation; ordinary polling is not MISP server-to-server sync.

Export format is also a selection policy

- Ordinary JSON can include unpublished contextual attributes.
- Detection formats can default to published and to_ids records.
- Set publication, to_ids and warninglist policy explicitly.
- A text export is a string, not a list of typed attributes.

Export format is also a selection policy
- Ordinary JSON can include unpublished contextual attributes.
- Detection formats can default to published and to_ids records.
- Set publication, to_ids and warninglist policy explicitly.
- A text export is a string, not a list of typed attributes.

Use pythonify=True with JSON in this course. Do not assume changing only return_format preserves the selected records. For a deliberate detection exercise use return_format="text", type_attribute="domain", published=1, to_ids=1, enforce_warninglist=True and pythonify=False, with an approved tag policy. The private unpublished fixture should not produce matching default detection output. Warninglists can remove rows after pagination. Synthetic .example output is not a production blocklist.

Build, save and verify

Build a private synthetic event offline

```
from pymisp import MISPEvent

event = MISPEvent()
event.info = "PyMISP 2.5 training / synthetic"
event.distribution = 0
event.threat_level_id = 4
event.analysis = 0
event.published = False
```


Build a private synthetic event offline

```
from pymisp import MISPEvent

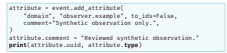
event = MISPEvent()
event.info = "PyMISP 2.5 training / synthetic"
event.distribution = 0
event.threat_level_id = 4
event.analysis = 0
event.published = False
```

Adapted from the MISPEvent cells in the upstream FullOverview and archived Introduction notebook. Explicit distribution 0 means your organisation only. Threat level 4 is Undefined; do not confuse that enumeration with analysis or distribution. Published False is a field on the new draft, not a request to publish it. examples/models.py adds a unique run token and makes no server request.

Add a contextual attribute to the local model

```
attribute = event.add_attribute(
    "domain", "observer.example", to_ids=False,
    comment="Synthetic observation only.",
)
attribute.comment = "Reviewed synthetic observation."
print(attribute.uuid, attribute.type)
```


Add a contextual attribute to the local model



Use reserved .example names and documentation IPs rather than copied malware indicators from older notebooks. to_ids=False means this is contextual data, not a deployment-ready detection assertion. event.add_attribute changes only the local event. misp.add_attribute sends a server request to an existing event. Preserve the distinction between a model helper and a client method. UUIDs support identity; names and values alone do not.

Group related observations in a template-backed object

```
from pymisp import MISPObject
obj = MISPObject("domain-ip", strict=True,
                 standalone=False)
obj.add_attribute("domain", value="host.example",
                  to_ids=False)
obj.add_attribute("ip", value="192.0.2.10",
                  to_ids=False)
event.add_object(obj)
```


Group related observations in a template-backed object

from pymisp import MISPObject
obj = MISPObject("domain-ip", strict=True,
standalone=False)
obj.add_attribute("domain", value="host.example",
to_ids=False)
obj.add_attribute("ip", value="192.0.2.10",
to_ids=False)
event.add_object(obj)

Adapted from the domain-ip example in FullOverview. The first argument to object.add_attribute is an object relation, not an arbitrary attribute type. The installed PyMISP wheel bundles templates, which can differ from the server's template catalogue. Strict construction plus obj.to_json(strict=True) checks local template requirements; it does not prove server acceptance or analytical truth. See the object-template course for identifiers, required/requiredOneOf fields and updates. Binary/file generators need optional extras; they are outside this workshop.

Inspect JSON and round-trip the local model

```
encoded = event.to_json(indent=2)
restored = MISPEvent()
restored.load(encoded)
assert restored.uuid == event.uuid
print(len(restored.attributes), len(restored.objects))
```


Inspect JSON and round-trip the local model



to_json returns a JSON string; to_dict returns a mapping whose values may still include model/date objects. Do not json.dumps a JSON string and accidentally double-encode it. PyMISP's client serialises models for API calls. Successful local round-trip preserves structure and identity but does not establish server validation, permissions or persistence. The offline tests check the actual reference target and strict object-template requirements as well.

Persist once, then use the server response

```python
from examples.common import expect_model

created = expect_model(
    misp.add_event(event, pythonify=True), MISPEvent)
verified = expect_model(
    misp.get_event(created.uuid, pythonify=True), MISPEvent
    )
```


Persist once, then use the server response

from examples.common import expect_model

created = expect_model(
    misp.add_event(event, pythonify=True), MISPEvent)
verified = expect_model(
    misp.get_event(created.uuid, pythonify=True), MISPEvent
)

The workshop starts from a newly generated draft and only modifies its own confirmed fixture. The response supplies server identity and accepted values; refetch to verify the outcome. Local event.add_object versus remote misp.add_object are different operations, as are local attribute.add_tag and remote misp.tag. Do not automatically retry an ambiguous POST: a timeout can occur after the event was created. Inspect the intended draft UUID before retrying. Publishing is a separate operation and is deliberately absent.

Update a known attribute and verify its identity

```
from pymisp import MISPAttribute
attribute = verified.attributes[0]
attribute.comment = "Reviewed in the workshop."
updated = expect_model(
    misp.update_attribute(attribute, pythonify=True),
    MISPAttribute,
)
assert updated.uuid == attribute.uuid
```


Update a known attribute and verify its identity

from pymisp import MISPAttribute
attribute = verified.attributes[0]
attribute.comment = "Reviewed in the workshop."
updated = expect_model(
misp.update_attribute(attribute, pythonify=True),
MISPAttribute,
)
assert updated.uuid == attribute.uuid

Refetch before editing and make the narrowest intended change. Do not replay an old full event dump as a routine update or reuse stale timestamps as if they were optimistic-concurrency locks. The released client serialises edited model fields and handles timestamps; still verify persisted values. update_attribute can return a proposal for a non-owned event, so require MISPAttribute rather than assuming any response means the original attribute changed. Check the refetched comment and to_ids state.

Reference an attribute from an object

```
obj.add_reference(attribute.uuid, "related-to",
                  "Synthetic teaching relation.")
# Add this before saving the event in the workshop.
```

- Keep the referenced UUID and relationship meaning explicit.
- A local reference is not a correlation or Galaxy attachment.

Reference an attribute from an object
obj.add_reference(attribute.uuid, "related-to",
"Synthetic teaching relation.")
Add this before saving the event in the workshop.
Keep the referenced UUID and relationship meaning explicit.
A local reference is not a correlation or Galaxy attachment.

The executable fixture creates the reference before add_event and verifies it after refetching the saved event. standalone=False retains object references when embedding the object in a full event dump. Cross-event references and references to unseen records have further server constraints and are not exercised here. Do not equate structural validity with evidence that a relationship is true.

Tag at the intended scope

```
from examples.common import expect_success

expect_success(misp.tag(created.uuid, "tlp:clear"))
expect_success(misp.tag(attribute.uuid, "tlp:clear",
                        local=True))
```


Tag at the intended scope
from examples.common import expect_success
expect_success(misp.tag(created.uuid, "tlp:clear"))
expect_success(misp.tag(attribute.uuid, "tlp:clear",
local=True))

These are remote operations. The operator must make tlp:clear available and the writer needs tagging permission. The first is an event tag; the second is a directly attached local attribute tag. Refetch and inspect the local flag rather than inferring scope from a rendered inherited tag. A handling tag is not a distribution setting or an access-control grant. This private training event remains organisation-only and unpublished regardless of its TLP tag. Tag creation permissions are separate from attachment permissions.

Sightings and proposals have distinct meanings

- `MISPSighting` plus `add_sighting` records an observation.
- Target the intended attribute UUID, not a broad shared value.
- Proposal methods create suggested changes for review.
- A returned proposal does not confirm an original was edited.

Sightings and proposals have distinct meanings
- `MISPSighting` plus `add_sighting` records an observation.
- Target the intended attribute UUID, not a broad shared value.
- Proposal methods create suggested changes for review.
- A returned proposal does not confirm an original was edited.

The workshop submits type 0 with the explicit source Synthetic PyMISP training sighting to its own attribute. It is a teaching record, not an external sensor observation. Type 1 denotes false positives and type 2 expiration; select deliberately. Single-attribute add/update/delete methods in this release can fall back to proposal methods for specific permission-denied responses. Bulk-add results also need per-item inspection. Other capabilities include analyst notes, event reports and feeds; consult current methods rather than assuming retired discussion/template APIs remain available.

Run the disposable write workshop deliberately

```
python -m examples.workshop --run
```

- Create a private event, attributes, object and reference.
- Edit, tag, add a sighting and refetch the result.
- Query bounded pages restricted to the new event UUID.
- Delete only that fixture and verify it is absent.



Use an isolated lab and an ordinary permitted writer. The operator prepares tlp:clear and the domain-ip template. The script does not accept an arbitrary existing event UUID for deletion. Its finally block attempts cleanup of the confirmed new event even if verification fails. If creation or cleanup is uncertain, inspect the lab before rerunning. This is a mutating exercise, unlike the default notebook and read-only CLI. Administrative provisioning is not a learner requirement.

Handle failures and diagnostics without losing meaning

- Separate transport failure, API error and valid empty result.
- Check saved state before retrying an uncertain write.
- Keep keys, private payloads and verbose logs out of notebooks.
- Record versions, query policy and validation limits.

Handle failures and diagnostics without losing meaning
- Separate transport failure, API error and valid empty result.
- Check saved state before retrying an uncertain write.
- Keep keys, private payloads and verbose logs out of notebooks.
- Record versions, query policy and validation limits.

PyMISP uses Python logging and can include server bodies or request data in diagnostics, especially in DEBUG mode. Do not enable debug=True or dump exceptions/responses into shared artifacts on sensitive systems. A timeout is not proof that a write failed, and a boolean success-looking field alone is not proof of the intended state. Use a deliberate retry/backoff policy for read operations and endpoint-specific idempotency/reconciliation for writes. The sample CLI gives concise errors; local protected diagnostics still need review.

Use the notebook as an experiment

- Default: create, inspect and round-trip local models.
- Optional live reads: compare raw and typed attribute UUIDs.
- Separate opt-in: execute the disposable write workshop.
- Clear outputs before sharing; retain source provenance.

Use the notebook as an experiment
- Default: create, inspect and round-trip local models.
- Optional live reads: compare raw and typed attribute UUIDs.
- Separate opt-in: execute the disposable write workshop.
- Clear outputs before sharing; retain source provenance.

PyMISP-workshop.ipynb uses the executable helpers rather than duplicating a second client implementation. MISP_LIVE=1 enables server reads; MISP_LAB_WRITE=1 additionally enables the disposable write cell. Neither is enabled by default. Configure credentials outside the notebook process and review every cell before opting in. The upstream notebooks provided the model/search teaching sequence; stale setup, real-world samples, automatic publication and admin examples were not carried into this exercise.

Make the integration reproducible

- Pin tested versions and preserve UUID-based identity.
- Separate local construction from remote persistence.
- Keep scope, selection policy and completion explicit.
- Use the README, executable examples and REST API course.

Make the integration reproducible
- Pin tested versions and preserve UUID-based identity.
- Separate local construction from remote persistence.
- Keep scope, selection policy and completion explicit.
- Use the README, executable examples and REST API course.

The README records the source audit, notebook cell provenance, validation results and limits. The earlier lesson, notes, images and published PDF are archived outside standard training. The new material does not claim a production sync engine, full PyMISP upstream CI or every MISP 2.5 patch/role combination. Consult the pinned client implementation and deployed OpenAPI for operations beyond the workshop. Contributions and issue reports are welcome in MISP/PyMISP and MISP/misp-training.